Lecture 10 Relational Database Design BCNF

United International College

Outline

- Features of Good Relational Design
- Atomic Domains and First Normal Form
- Functional Dependency Theory
- BCNF
- 3rd Normal Form
- Multivalued Dependencies and Decomposition
- Database-Design Process

Motivation of Decompositions

- To remove the redundancy in the database, we intend to decompose the schema into normal forms.
- The ideal decomposition satisfies the following conditions
  - the redundancy is minimized (recall the example given in the section “Good Design”);
  - the decomposition is lossless; and
  - the dependencies are preserved.

Lossless-join Decomposition

- For the case of $R = R_1 \cup R_2$, a decomposition of R into R_1 and R_2 is lossless join if and only if **at least one** of the following dependencies is in F^+:
  - $R_1 \cap R_2 \rightarrow R_1$
  - $R_1 \cap R_2 \rightarrow R_2$

Dependency Preservation

- Let the schema R is decomposed into $R_1, R_2, \cdots, R_n$.
- Let F_i be the subset of dependencies F^+ that only includes attributes in R_i for $1 \leq i \leq n$.
- The decomposition is **dependency preserving**, if
$$(F_1 \cup F_2 \cup \cdots \cup F_n)^+ = F^+$$
- If the decomposition is not dependency preserving, then checking updates for violation of functional dependencies may require computing joins, which is expensive.

Example

$R = \{A, B, C\}$
$F = \{A \rightarrow B, B \rightarrow C\}$

Can be decomposed in two different ways

- $R_1 = \{A, B\}$, $R_2 = \{B, C\}$
  - Lossless-join decomposition
    $R_1 \cap R_2 = \{B\}$ and $B \rightarrow BC$
  - Dependency preserving
- $R_1 = \{A, B\}$, $R_2 = \{A, C\}$
  - Lossless-join decomposition
    $R_1 \cap R_2 = \{A\}$ and $A \rightarrow AB$
  - Not dependency preserving
    (cannot check $B \rightarrow C$ without computing $R_1 \bowtie R_2$)

Boyce-Codd Normal Form

- A relation schema R is in **BCNF** with respect to a set of functional dependencies F if for all functional dependencies in F^+ of the form

$$\alpha \rightarrow \beta$$

where $\alpha \subseteq R$ and $\beta \subseteq R$, at least one of the following holds:

1. $\alpha \rightarrow \beta$ is trivial (i.e., $\beta \subseteq \alpha$);
2. α is a superkey for R.

- If R is not in BCNF because of the functional dependency $\alpha \rightarrow \beta$, we say $\alpha \rightarrow \beta \in F$ to R **violates** BCNF.
- Example schema *not* in BCNF:

$$course\_info = (\underline{c\_name, p\_code}, credits, domain, c\_number)$$

because $c\_name \rightarrow credits, domain, c\_number$ holds on $course\_info$ but c_name is not a superkey.

Example

$R = \{A, B, C\}$

$F = \{A \rightarrow B, B \rightarrow C\}$

$Key = \{A\}$

- R is not in BCNF because $B \rightarrow C$ violates BCNF.
- Decomposition

$$R_1 = \{A, B\},\ F_1 = \{A \rightarrow B\}$$

$$R_2 = \{B, C\},\ F_2 = \{B \rightarrow C\}$$

  - R_1 and R_2 are in BCNF. The decomposition is
  - lossless-join decomposition and
  - dependency preserving.

BCNF Decomposition Algorithm

Algorithm Decompose R in BCNF

1. $result = \{R_0\} = \{R\}$
2. $i = 0$
3. compute F^+
4. **while** $\alpha \rightarrow \beta \in F_j$ to $R_j \in result$ violates BCNF for $0 \leq j \leq i$ **do**
5. $\quad R_{i+1} = \alpha \cup \beta$
6. $\quad F_{i+1} = \{\alpha \rightarrow \beta | \alpha \rightarrow \beta \in F^+$ and $\alpha, \beta \subseteq R_{i+1}\}$
7. $\quad R_{i+2} = R_j \setminus (\beta \setminus \alpha)$
8. $\quad F_{i+2} = \{\alpha \rightarrow \beta | \alpha \rightarrow \beta \in F^+$ and $\alpha, \beta \subseteq R_{i+2}\}$
9. $\quad result = \left(result \setminus \{R_j\}\right) \cup \{R_{i+1}\} \cup \{R_{i+2}\}$
10. $\quad i = i + 2$
11. **end while**
12. **return** $result$

- "\" is set minus, same as "exclude" or "except".
- $result$ is a set of schemas, not only one schema.
- The loop on line 4 is executed if a schema R_j with a FD $\alpha \rightarrow \beta$ violates BCNF.

BCNF Decomposition Algorithm

This is the decomposition algorithm in a high-level language.

- The return of the algorithm is always a set of schemas. Initially, the set has only one schema, *R* itself.
- In each iteration, if there is a functional dependency $\alpha \rightarrow \beta$ makes the schema R_j not in BCNF, decompose R_j into
  - the new schema 1:
    $$R_{i+1} = \alpha \cup \beta$$
  - and the new schema 2:
    $$R_{i+2} = R_j \setminus (\beta - \alpha)$$
  - The functional dependency set for R_{i+1} and R_{i+2} consist of the functional dependencies with both sides all in R_{i+1} and R_{i+2} respectively.
  - Replace the old schema by the two new schemas.

BCNF Example

- $R = \{\underline{person\_id, ISBN}, person\_name, book\_title, author\}$
  $F = \{person\_id \rightarrow person\_name,$
  $ISBN \rightarrow book\_title, author\}$
- Iteration 1: $person\_id \rightarrow person\_name$ violates BCNF. Decompose R into
  - $R_1 = \{\underline{person\_id}, person\_name\}$
    $F_1 = \{person\_id \rightarrow person\_name\}$
  - $R_2 = \{\underline{person\_id, ISBN}, book\_title, author\}$
    $F_2 = \{ISBN \rightarrow book\_title, author\}$

BCNF Example

- $R_1 = \{\underline{person\_id}, person\_name\}$

  $F_1 = \{person\_id \rightarrow person\_name\}$
- $R_2 = \{\underline{person\_id, ISBN}, book\_title, author\}$

  $F_2 = \{ISBN \rightarrow book\_title, author\}$

- Iteration 2: $ISBN \rightarrow book\_title, author$ violates BCNF. Decompose R_2 into
  - $R_3 = \{\underline{ISBN}, book\_title, author\}$

    $F_3 = \{ISBN \rightarrow book\_title, author\}$
  - $R_4 = \{\underline{person\_id, ISBN}\}$

    $F_4 = \{\}$

- Result R_1, R_3, and R_4.

Pros and Cons of BCNF

- Advantages:
  - Redundancies are minimized.
  - BCNF decomposition is a lossless-join decomposition.
- Disadvantage:
  - The decomposition is not always dependency preserving.
  - For example,

    $R = \{J, K, L\}$

    $F = \{JK \rightarrow L, L \rightarrow K\}$

    Candidate keys are $\{J, K\}$ and $\{J, L\}$.
  - R is not in BCNF because of $L \rightarrow K$.
  - The algorithm decomposes R into $\{J, L\}$ and $\{L, K\}$ which fails to preserve $JK \rightarrow L$.
  - Testing $JK \rightarrow L$ requires a join.

End of Lecture 10